

Splitting ARENA Notebooks into Self-Contained Drills

A worked case study: where the split was genuinely hard, and how we did it anyway

Delta Drills | delta-drills.vercel.app

June 2026

The claim under test

“While there are some existing efforts to address this issue (MiniARENA, CAMBRIA, ARBOx3), ARENA notebooks are designed to be completed in continuous segments, making it difficult to split and effectively run the notebooks in shorter, self-contained sessions.”

The premise is correct for *stock* ARENA: each notebook is a continuous narrative in which exercise N depends on definitions, imports and fixtures established in cells $1 \dots N-1$. Pull any single exercise out and it will not run.

This document is the counter-example. Delta Drills decomposed ARENA Chapter 0 into **2,538 standalone notebooks** — 582 single-atom drills, 180 composite drills, and per-question solution notebooks — where *every exercise carries its own setup* and runs in a fresh Colab kernel with nothing executed before it. The hard part wasn’t volume; it was the topics where the continuous narrative *is* the point — the autograd engine, the convolution-as-strided-view, the DCGAN decoder. Those are the examples below.

1 How a split exercise is made self-contained

The mechanism is the same in every drill, and it is what defeats “continuous segments”: **each notebook re-declares the minimal scaffolding it needs, inline, inside its own test harness.** It never imports the rest of the chapter.

A drill notebook is a fixed 6-part skeleton:

1. **Setup cell** — imports + `t.manual_seed(0); np.random.seed(0)`. Deterministic, self-contained.
2. **Auth cell** — `DD_TOKEN, DD_ATOM_IDS, DD_SUBTOPICS`, backend URL.
3. **“How these atoms compose”** — the prose that replaces the missing notebook context.
4. **Exercise stub** — a single function with `raise NotImplementedError`.
5. **Inline test harness** — re-declares the local `Recipe / MiniTensor / back_funcs` etc. that the real ARENA notebook would have built over many cells, then asserts correctness.
6. **Completion beacon** — POSTs to `/api/practice/arena-rating`, updating BKT mastery for

every atom the drill exercises.

The crucial line is in the grader (mirrored inside every notebook's test): seed *then* run per-exercise setup, so the learner's function executes against a known fixed state with no dependence on prior cells.

```
np.random.seed(0)                # known state
exec(case["setup_code"], globals()) # ONLY this exercise's fixtures
actual = eval(case["call"], globals()) # run solve() in isolation
```

2 Examples where the split was genuinely hard — and we did it

The autograd chapter (ARENA [0.4] Backprop) is the hardest case: it builds a working mini-autograd engine where *everything depends on everything* — a `Tensor` wrapper, a `Recipe` of how each value was computed, a topological sort, and a reverse loop that dispatches per-argument backward functions. You cannot test the reverse loop without the graph; you cannot build the graph without the wrapper. Stock ARENA does this across one long notebook. We split it into atoms that each *rebuild just enough of the engine to stand alone*.

2.1 The full reverse pass, in one isolated cell

cx25 — seed reverse pass with ones_like; pop-and-dispatch loop. [Open in Colab](#) ↗

This is the heart of the whole chapter. The notebook re-declares a `Recipe` dataclass, a `MiniTensor` class, and two `multiply_back` functions *inside the test*, so the reverse-mode loop can be exercised with zero upstream cells:

```
def cx25_backprop(end_node, end_grad, sorted_graph, back_funcs):
    if end_grad is None:
        seed = t.ones_like(end_node.array)          # d(end.sum())/d(end)
    else:
        assert end_grad.array.shape == end_node.array.shape
        seed = end_grad.array
    grads = {id(end_node): seed}
    for node in sorted_graph:                        # reverse-pass driver
        grad_out = grads.pop(id(node), None)
        if grad_out is None: continue
        if node.recipe is None:                     # leaf -> .grad
            node.grad = grad_out if node.grad is None else node.grad + grad_out
            continue
        for argnum, parent in node.recipe.parents.items():
            back = back_funcs[(node.recipe.func, argnum)]
            gp = back(grad_out, node.array, *node.recipe.args, **node.recipe.kwargs)
            grads[id(parent)] = grads.get(id(parent), 0) + gp
```

Why it is hard to split, and why this split is good: the seed step and the loop step are *inseparable* in ARENA — the loop can't start without a seed in `grads`, and the seed has no purpose without the loop. Rather than fake that by stubbing the engine, the drill keeps both atoms in one function and supplies a tiny but *real* graph in the test. The learner writes the exact code they'd write in ARENA.

2.2 Topological sort with cycle detection

cx19 — three-colour DFS: deps-first order *and* ValueError on a cycle. [Open in Colab](#)



A single DFS walk carries two responsibilities — two atoms — that ARENA never separates: the `perm/temp` colour sets (cycle detection) and the post-order append (deps-first ordering). The test feeds a linear chain, a diamond DAG (node must appear once), and a self-loop (must raise), so passing proves both atoms compose correctly.

```
def cx19_topo_sort_with_cycle_check(root, get_children):
    result, perm, temp = [], set(), set()
    def visit(node):
        nid = id(node)
        if nid in perm: return # finished subtree, fine
        if nid in temp: raise ValueError(f'cycle at {node!r}') # back-edge
        temp.add(nid)
        for child in get_children(node): visit(child)
        temp.remove(nid); perm.add(nid)
        result.append(node) # deps-first append
    visit(root); return result
```

2.3 Wiring a brand-new op into the engine

cx1 — register a forward op so it builds a Recipe and back-propagates. [Open in Colab](#)



cx30 — unbroadcast inverts NumPy broadcasting in the backward pass. [Open in Colab](#)



These two are the “glue” atoms that, in stock ARENA, are impossible to test until the entire engine exists. Splitting them out required re-creating a minimal differentiable-op registry in the test cell — which is exactly the elegant part: the drill proves you understand the contract (**forward** builds a recipe; **back** replays it; gradients un-broadcast back to parameter shape) without the 400-line notebook around it.

3 Elegant splits: hard core problem, clean two-line drill

These are cases where the underlying ARENA exercise is conceptually heavy, but the atomised drill is short and sharp — the split *distils* the difficulty instead of diluting it.

3.1 Conv2d forward as an as_strided view + einsum

cx2 — the whole convolution in two lines. [Open in Colab](#)



ARENA’s convolution exercise is one of the chapter’s most feared. The atomised version reduces it to its two load-bearing ideas — a strided patch view and a single einsum contraction:

```
def cx2_conv2d_via_einsum(x, w):
    B, C_in, H, W = x.shape
    C_out, _, KH, KW = w.shape
    H_out, W_out = H - KH + 1, W - KW + 1
    sB, sC, sH, sW = x.stride()
    patches = x.as_strided(size=(B, C_in, H_out, W_out, KH, KW),
```

```

        stride=(sB, sC, sH, sW, sH, sW))    # spatial stride twice
    return einops.einsum(patches, w, 'b c h w kh kw, o c kh kw -> b o h w')

```

The elegance is in the stride tuple: the spatial strides appear *twice* because the same 2-D step walks both the patch origin and the inside of each patch. The drill isolates exactly that insight. (Companion drill cx1 [Open in Colab](#) ↗ covers deriving the output shape that sizes the view.)

3.2 Batched ray/triangle solve

cx2 — stack columns, then one batched `linalg.solve`. [Open in Colab](#) ↗

```

def cx2_solve_from_columns(cols, b):
    A = t.stack(cols, dim=-1)    # NEW trailing axis -> (K, n, n)
    return t.linalg.solve(A, b)  # K LU factorisations fused into one BLAS call

```

Two atoms, two lines. The drill hinges on the `stack` vs `cat` choice — `cat(dim=-1)` would silently give `(K, n*n)` and either crash or, worse, return garbage. (Companion cx4 [Open in Colab](#) ↗ adds the singular-slice patch that real ray tracing needs.)

3.3 The DCGAN decoder, as one `nn.Module`

cx4 — latent $z \rightarrow$ Linear projection $\rightarrow$ reshape $\rightarrow$ ConvT/BN/ReLU upsample. [Open in Colab](#) ↗

```

class Decoder(nn.Module):
    def __init__(self, latent_dim, base_channels, base_hw):
        super().__init__()
        self.base_c, self.base_hw = base_channels, base_hw
        self.proj = nn.Linear(latent_dim, base_channels * base_hw * base_hw) # bare, no act
        self.block = nn.Sequential(
            nn.ConvTranspose2d(base_channels, base_channels // 2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(base_channels // 2),
            nn.ReLU(inplace=True))

```

A generative-model component that ordinarily only makes sense inside a full training notebook becomes a self-contained module-construction drill. (Companion cx1 [Open in Colab](#) ↗ drills the ConvT/BN/activation block on its own.)

3.4 Optimizer internals

cx2 — momentum SGD `__init__`: materialise params *and* per-param velocity buffer. [Open in Colab](#) ↗

cx4 — update the velocity buffer in place via `copy_`. [Open in Colab](#) ↗

The optimizer loop is another ARENA exercise that only “works” once the whole training script exists. These drills isolate the two subtle bits — buffer allocation at init, and in-place update — each runnable against a couple of throwaway tensors.

4 Full example index

Every link opens a live Colab notebook from the AkiraTheSquid/Delta-Drills repo (main branch). Signed-in students get their own fork via the account GitHub-username field, so Colab “Save a copy

in GitHub” persists their work.

Domain	Drill	Link
Autograd	cx25 seed + pop-and-dispatch reverse pass	Open in Colab ↗
Autograd	cx19 DFS toposort + cycle detection	Open in Colab ↗
Autograd	cx1 wire a new op into autograd	Open in Colab ↗
Autograd	cx30 unbroadcast inverts broadcast	Open in Colab ↗
CNN	cx2 Conv2d via as_strided + einsum	Open in Colab ↗
CNN	cx1 as_strided view sized by conv output shape	Open in Colab ↗
Ray tracing	cx2 stack columns then batched solve	Open in Colab ↗
Ray tracing	cx4 patch singular slices then solve	Open in Colab ↗
Optimizers	cx2 momentum SGD init + velocity buffers	Open in Colab ↗
Optimizers	cx4 velocity buffer update via copy_	Open in Colab ↗
VAE/GAN	cx4 latent projection then ConvT block	Open in Colab ↗
VAE/GAN	cx1 DCGAN G block as nn.Module subclass	Open in Colab ↗
Einops	cx2 outer-product two 1-D grids via repeat	Open in Colab ↗

The full library is browsable in-app at delta-drills.vercel.app (Practice tab) and in the repo under `arena-procedural-drills/`.

5 Method: how the splits were extracted

The split was a pipeline, not a manual port. Each stage *recycled the artifact built for testing the previous stage* — which is what made it cheap to scale to thousands of notebooks.

1. Atomise the curriculum

ARENA’s exercises were decomposed into a concept graph of **207 drillable atoms** (e.g. `as-strided-windowing`, `dfs-three-set-toposort`, `end-grad-default-ones-like`), with prerequisite and encompassing

edges. Each atom is a single testable skill. Composite drills then exercise 2–3 atoms *together*, because some atoms (the seed and the reverse loop) are inseparable in real code.

2. Author standalone scaffolding per atom

For each atom a notebook was generated carrying its own imports, deterministic seed, a function stub, and — critically — an **inline test harness that rebuilds the minimal engine** (Recipe, MiniTensor, back_funcs, helper graph nodes) the exercise needs. This is the step that breaks “continuous segments”: the context that ARENA accumulates across cells is re-created locally, in miniature, in one cell.

3. Recycle the grader as the validator

The same harness that scores a student’s submission at runtime (`code_runner.run_function_tests: seed → run setup_code → call solve()`) was reused at *build time* to validate every authored solution. A drill ships only if the reference solution passes its own test under the live grader. The test fixture, the validation oracle, and the production grader are one object — not three.

This recycling surfaced a real, load-bearing quirk: the grader seeds the RNG *then* runs `setup_code` (which consumes the stream), so a correct `solve()` must *reseed internally* before drawing. Because the validator *is* the grader, every authored solution was forced to obey the same contract students face — no drift between “passes in authoring” and “passes in the app”.

4. Build solution + hint siblings from the same source

For each problem notebook a `.solution.ipynb` sibling was generated by lifting the code out of the `<details><summary>Show solution</summary>` block into the `NotImplementedError` stub (761/762 drills), then re-running the grader to confirm the solution notebook itself executes clean. Per-drill nudge hints were authored and validated the same way. The “Show answer” / “Show hint” buttons in the app point at these siblings, routed to the student’s Colab fork.

5. Wire completion back to mastery

Each notebook ends with a beacon POSTing to the backend, which updates Bayesian Knowledge Tracing mastery for every atom the drill exercises and unlocks downstream atoms through the prerequisite graph. The split isn’t just *runnable* in a short session — it *closes the loop* on the adaptive engine.

Verdict

The claim’s premise holds for stock ARENA; its implied limitation does not hold against this system. Delta Drills shipped the self-contained-session split the claim treats as the open problem — including on the chapters (autograd, convolution, DCGAN) where the continuous narrative is hardest to break — with reproducible per-exercise setup, isolated grading reused as the build-time validator, and mastery wired back through the concept graph. The honest framing: ARENA is hard to split *by default*; we solved it with atomisation plus self-contained scaffolding, not by wishing the dependency away.